

Python Conditional Statements

for Beginners

Learn how to make your programs think and decide what to do next

if else



==



< >



!=

What Are Conditional Statements?

Decision Making in Code

Conditional statements are fundamental tools that allow your code to make decisions based on whether certain conditions are true or false.



They enable programs to execute different blocks of code based on specific conditions.



This ability to control the flow of a program is crucial for creating dynamic and responsive applications.

Real-World Analogy



"If it is raining, I will take an umbrella."

This simple thought process is exactly what conditional statements enable computers to do.

How Conditional Statements Work

```
if condition:  
    print("Do this")  
else:  
    print("Do that")
```



● True → Execute first block

● False → Execute else block

Understanding Conditions and Operators

≠ Comparison Operators

Comparison operators compare two values and return a Boolean result (**True** or **False**).

`==` Equal to
`5 == 5` → **True**

`!=` Not equal to
`5 != 10` → **True**

`>` Greater than
`10 > 5` → **True**

`<` Less than
`5 < 10` → **True**

```
# Example in Python
if x > y:
    print("x is greater than y")
```

🔗 Logical Operators

Logical operators combine multiple conditional statements to form more complex conditions.

and Both conditions must be True
True and True → **True**
True and False → **False**

or At least one condition must be True
True or False → **True**
False or False → **False**

not Reverses the Boolean value
not True → **False**
not False → **True**

The Basic if Statement

How if Statements Work

The **if** statement executes a block of code **only if** a given condition is true.

💡 Think of it as saying, "IF this is true, THEN do this."

Syntax:

```
if condition:  
    # Code to execute if condition is True
```

⚠️ Important: The colon (:) is required!

Examples

Example 1:

```
number = 10  
if number > 0:  
    print("The number is positive.")  
    print("This statement always executes.")
```

Output:

The number is positive.

This statement always executes.

The if-else Statement

Two-way Decision Making

The `if-else` statement chooses between two actions based on a condition.

- ✓ If condition is **True**, execute code after `if`.
- ✗ If condition is **False**, execute code after `else`.
- 🔗 Exactly one code block will be executed.

Syntax

```
if condition:
    # Code to execute if True
    pass
else:
    # Code to execute if False
    pass
```

Example: Even or Odd

```
number = 7
if number % 2 == 0:
    print(f"{number} is even.")
else:
    print(f"{number} is odd.")
```

💡 **Output:** "7 is odd."

The if-elif-else Statement

Handling Multiple Conditions

The **if-elif-else** statement allows you to check multiple multiple conditions in a clear way.

- ✓ Python checks each condition **one by one**, from top to top to bottom.
- 💡 As soon as it finds a condition that is **True**, it executes executes the code block.
- 🔗 If none are true, the **else** block (if present) is executed. executed.

Visual Example

```
score = 85
if score >= 90:
    print("Grade: A")
elif score >= 80:
    print("Grade: B")
elif score >= 70:
    print("Grade: C")
elif score >= 60:
    print("Grade: D")
else:
    print("Grade: F")
```

85 >= 90

False



85 >= 80

True



85 >= 70

Skipped

85 >= 60

Skipped

Output: "Grade: B"

Advanced Conditional Structures

Nested if Statements

An **if statement** placed inside another if statement.

- ✓ **Refined control flow** for complex decisions
- ✓ **Multiple conditions** evaluated sequentially

```
if outer_condition:
    if inner_condition:
        execute_inner_block
    else:
        execute_outer_else
else:
    execute_final_else
```

 **Remember:** Only executes innermost block when all outer conditions are True.

Ternary Operator

A concise way to write simple if-else statements in a single line.

- ✓ **Compact syntax** for simple conditions
- ✓ **Readable code** for straightforward assignments

```
if age >= 18:
    status = "Adult"
else:
    status = "Minor"
```

```
status = "Adult" if age >= 18
else "Minor"
```

 **Caution:** Avoid overuse or nesting too many ternary operators as it can reduce readability.

When to Use Each Structure

-  **Nested if:** Complex logic with multiple dependencies
-  **Ternary:** Simple assignments that can be expressed concisely

Practice and Summary

✓ Key Takeaways

- Conditional statements allow programs to make decisions
- Basic if statement executes code when a condition is true
- if-else provides two paths for true and false conditions
- if-elif-else handles multiple conditions sequentially
- Comparison and logical operators form the foundation

Practice Resources



W3Schools

Beginner-friendly exercises on conditional statements



Programiz

Practical examples with interactive code editor



GeeksforGeeks

Challenging problems to test your understanding

Remember the Syntax

```
if condition:  
    print("True")
```

```
if condition:  
    print("True")  
else:  
    print("False")
```

```
if condition1:  
    print("1")  
elif condition2:  
    print("2")  
else:  
    print("3")
```